



北京大学

算分第六次作业

8.1

图: $G_1, G_2 \Rightarrow A, B \Rightarrow a_{ij}, b_{ij}$

解: (1) 借用图的邻接矩阵表示:

算法 for i in $1 \sim n$:

for j in $1 \sim n$ and $j \neq i$:

if $a_{ij} \neq b_{ij}$

return 不是

return 是补图

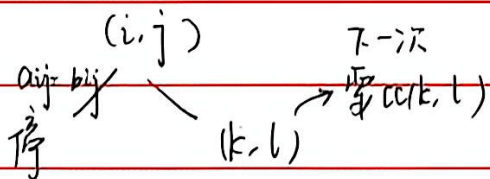
→ 这个算法有重复
可优化为

for j in $i+1 \sim n$

时间复杂度 $n(n-1)$

(2) 下界, $\frac{n(n-1)}{2}$

证明: 考虑决策树



注意所有 (i, j) 对都必须被比较

不同输入 $2^{\frac{n(n-1)}{2}}$ 种, 每个输入对应一种决策树路径

而这种二叉树 ($2^{\frac{n(n-1)}{2}}$ 个树叶) 深度至少为 $\log(2^{\frac{n(n-1)}{2}}) = \frac{n(n-1)}{2}$ □

即对应最坏时间复杂度

8.2

解: (1) 考虑分步迭代

$$P_0(x) = a_n$$

$$P_1(x) = a_{n-1} + P_0(x) \cdot x$$

$$P_n(x) = a_0 + P_{n-1}(x) \cdot x = P(x)$$



北京大学

可以验证每次迭代都是一个加法与一个乘法，时间复杂度为常数

∴ $n > 1$ 迭代后时间复杂度为 $O(n)$

12) 证明：只需考虑 $0 \sim n$ 这个系数，每个都需要至少处理一次。
否则遇到第 k 次系数不同时，求值算法若不处理第 k 次无法正确输出不同结果。□

8.4

解法一

11) 顺序归并：

$L_1 \quad L_2 \quad \dots \quad L_k$

✓

$O(2 \cdot \frac{n}{k})$

✓

$O(3 \cdot \frac{n}{k})$

...

$O(k \cdot \frac{n}{k})$

∴ 总时间复杂度为

$$O\left(\frac{k+0(k-1)}{2} \cdot \frac{n}{k}\right)$$

$$= O(kn)$$

(奇数则剩一组，留到下一轮)

12) 二分归并：两两分组归并，依此类推，直到剩 2 个表，最后归并

这样，每个表的元素被平均归并了 $\log k$ 次，每次归并表中元素比较 1 次

∴ 时间复杂度为 $O(\log k \cdot n)$

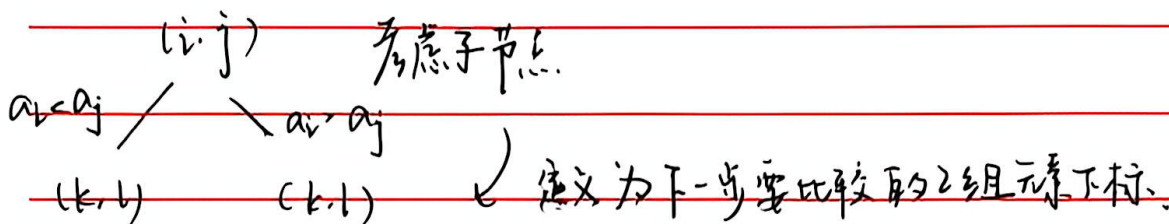


北京大学

13) 下界: $O(n \log k)$

证明: 构造决策树

节点 (i, j) 表示算法要比较元素 a_i, a_j .



考虑不同输入个数: $C_n^{\frac{n}{k}} C_{n-\frac{n}{k}}^{\frac{n}{k}} \dots C_{\frac{n}{k}}^{\frac{n}{k}} = \frac{n!}{[(\frac{n}{k})!]^k}$

$\Rightarrow$ 树叶节点数不小于 $\frac{n!}{[(\frac{n}{k})!]^k}$

树深度 $d \geq \lceil \log \frac{n!}{[(\frac{n}{k})!]^k} \rceil \geq n \log n - \frac{n}{k} \log(n/k)$
 $= \Theta(n \cdot \log k)$ $\square$

8-9. 解:

- 1) 算法: 找第 m 小数字 $\Rightarrow \text{select}(\frac{n}{\log n}) \Rightarrow O(n)$ return a_m
- ② 将剩余所有数与 a_m 比较, 挑出前 m 小 $\Rightarrow O(n)$ (遍历一遍)
- ③ 最后对前 m 数排序.

$$\begin{aligned}
 \text{复杂度 } O(m \log m) &= O\left(\frac{n}{\log n} \log\left(\frac{n}{\log n}\right)\right) \\
 &= O\left(\frac{n}{\log n} \cdot \log \log n + n\right) = O(n)
 \end{aligned}$$

综上, 复杂度为 $O(n)$



北京大学

12) 证明: $\therefore$ 要有序输出.

$\therefore$ 必须对 m 个数排序.

已知排序复杂度下界为 $O(m \log m)$

又 $m = \omega(n / \log n)$

$\therefore$ 复杂度下界 $= \Omega\left(\omega\left(\frac{n}{\log n} \cdot \log \frac{n}{\log n}\right)\right) = \omega\left(n - \frac{n \log \log n}{\log n}\right) = \omega(n)$

$\therefore$ 不存在 $O(n)$ 算法 $\square$.

8.10 证明:

设第 k 小数 a_k .

def: 决定性比较: 能够确定 a_k 与 x 的关系

e.g. $\exists y, (x > y, y > t)$
或 $(x < y, y \leq t)$

反之, 非决定性.

下面构造输入:

对 $\forall$ 算法 A .

尽量要多“非决定性”来使输入“差”.

具体而言

比 x, y : 若都被赋值 $x > a_k, y < a_k$ (这样 x, y 比无效),

若 $x > a_k, y$ 未被赋值 $\Rightarrow y < a_k$
 $x < a_k \quad y > a_k$

非决定性比较

这样的比较至少有 $c = \min(k-1, n-k)$ 个才能都被赋值, 或无法满足边赋值. 之后 $n-1$ 次决定性比较: (至少)

综上, 共 $n-1+c \geq n-1+\min\{k, n-k+1\}$ $\square$.